Web Services Assignment 2026

Fast API:

```
import os
import requests
from typing import Annotated
from fastapi import FastAPI, HTTPException, Path, Query
from pymongo import MongoClient
from pymongo.errors import PyMongoError
from prometheus_fastapi_instrumentator import Instrumentator

# Initialize FastAPI app and Prometheus instrumentation
app = FastAPI()
Instrumentator().instrument(app).expose(app)


# MongoDB connection
mongo_uri = os.getenv("MONGO_URI", "mongodb://localhost:27017")
mongo_db_name = os.getenv("MONGO_DB", "inventory_db")

client = MongoClient(mongo_uri)
db = client[mongo_db_name]
collection = db["products"]

# Helper function to remove MongoDB's _id field from documents
# This ensures that the API responses do not include the internal MongoDB
identifier.
def _strip_mongo_id(doc):
    if doc is None:
        return None
    if isinstance(doc, list):
        for item in doc:
            if isinstance(item, dict):
                item.pop("_id", None)
        return doc
    if isinstance(doc, dict):
        doc.pop("_id", None)
        return doc
    return doc


@app.get("/")
def home():
    return {
        "message": "Inventory API is running",
        "endpoints": [
```

```
            "/getSingleProduct/{product_id}",
            "/getAll",
            "/addNew",
            "/deleteOne/{product_id}",
            "/startsWith/{letter}",
            "/paginate/{start_id}/{end_id}",
            "/convert/{product_id}"
        ]
    }


@app.get("/getSingleProduct/{product_id}")
def get_single_product(product_id: str):
    try:
        product = collection.find_one({"ProductID": str(product_id)},
{"_id": 0})
        if not product:
            raise HTTPException(status_code=404, detail="Product not found")
        return _strip_mongo_id(product)
    except PyMongoError as e:
        raise HTTPException(status_code=500, detail=f"Database error:
{str(e)}")


@app.get("/getAll")
def get_all():
    try:
        products = list(collection.find({}, {"_id": 0}))
        return _strip_mongo_id(products)
    except PyMongoError as e:
        raise HTTPException(status_code=500, detail=f"Database error:
{str(e)}")


@app.post("/addNew")
def add_new(
    ProductID: Annotated[str, Query(min_length=1)],
    Name: Annotated[str, Query(min_length=1)],
    UnitPrice: Annotated[float, Query(gt=0)],
    StockQuantity: Annotated[int, Query(ge=0)],
    Description: Annotated[str, Query(min_length=1)]
):
    try:
        existing = collection.find_one({"ProductID": str(ProductID)})
        if existing:
            raise HTTPException(status_code=400, detail="ProductID already
exists")

        new_product = {
```

```
            "ProductID": str(ProductID),
            "Name": Name,
            "UnitPrice": str(UnitPrice),
            "StockQuantity": str(StockQuantity),
            "Description": Description
        }

        collection.insert_one(new_product)
        new_product.pop("_id", None)
        return {"message": "Product added successfully", "product":
new_product}

    except PyMongoError as e:
        raise HTTPException(status_code=500, detail=f"Database error:
{str(e)}")


@app.delete("/deleteOne/{product_id}")
def delete_one(product_id: str):
    try:
        result = collection.delete_one({"ProductID": str(product_id)})

        if result.deleted_count == 0:
            raise HTTPException(status_code=404, detail="Product not found")

        return {"message": f"Product {product_id} deleted successfully"}

    except PyMongoError as e:
        raise HTTPException(status_code=500, detail=f"Database error:
{str(e)}")


@app.get("/startsWith/{letter}")
def starts_with(letter: Annotated[str, Path(min_length=1, max_length=1)]):
    try:
        regex_pattern = f"^{letter}"
        products = list(
            collection.find(
                {"Name": {"$regex": regex_pattern, "$options": "i"}},
                {"_id": 0}
            )
        )
        return _strip_mongo_id(products)
    except PyMongoError as e:
        raise HTTPException(status_code=500, detail=f"Database error:
{str(e)}")


@app.get("/paginate/{start_id}/{end_id}")
```

```
def paginate(start_id: str, end_id: str):
    try:
        products = list(
            collection.find(
                {
                    "ProductID": {
                        "$gte": str(start_id),
                        "$lte": str(end_id)
                    }
                },
                {"_id": 0}
            ).sort("ProductID", 1).limit(10)
        )
        return _strip_mongo_id(products)
    except PyMongoError as e:
        raise HTTPException(status_code=500, detail=f"Database error:
{str(e)}")


@app.get("/convert/{product_id}")
def convert(product_id: str):
    try:
        product = collection.find_one({"ProductID": str(product_id)},
{"_id": 0})
        if not product:
            raise HTTPException(status_code=404, detail="Product not found")

        product = _strip_mongo_id(product)

        usd_price = float(product["UnitPrice"])

        # realtime Exchangerate API call to get current USD to EUR exchange
rate
        exchangerate_key = os.getenv("EXCHANGE_RATE_API_KEY", "").strip()
        if exchangerate_key:
            # if key specified, use the official endpoint that requires the
API key
            url = f"https://v6.exchangerate-
api.com/v6/{exchangerate_key}/latest/USD"
        else:
            # Fallback endpoint no key required
            url = "https://open.er-api.com/v6/latest/USD"

        exchange_response = requests.get(url, timeout=10)
        exchange_response.raise_for_status()
        exchange_data = exchange_response.json()

        rates = exchange_data.get("conversion_rates") or
exchange_data.get("rates") or {}
```

```
        eur_rate = rates.get("EUR")
        if eur_rate is None:
            raise HTTPException(status_code=500, detail="Exchange API error:
EUR rate not found")

        eur_price = round(usd_price * eur_rate, 2)

        return {
            "ProductID": product["ProductID"],
            "Name": product["Name"],
            "PriceUSD": usd_price,
            "ExchangeRateUSDtoEUR": eur_rate,
            "PriceEUR": eur_price
        }

    except requests.RequestException as e:
        raise HTTPException(status_code=500, detail=f"Exchange API error:
{str(e)}")
    except ValueError:
        raise HTTPException(status_code=500, detail="Invalid UnitPrice
stored in database")
    except PyMongoError as e:
        raise HTTPException(status_code=500, detail=f"Database error:
{str(e)}")
```

Unit Tests:

```
{
  "info": {
    "_postman_id": "e74edc58-9cb1-4bcf-b2b4-fd3c5a0f4f3b",
    "name": "Inventory API - Endpoint Tests",
    "schema":
"https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "item": [
    {
      "name": "Home",
      "request": {
        "method": "GET",
        "url": "{{baseUrl}}/"
      },
      "event": [
        {
          "listen": "test",
          "script": {
```

```
            "exec": [
              "pm.test('Status 200', function ()
{ pm.response.to.have.status(200); });",
              "pm.test('Has endpoints list', function () {",
              "  const data = pm.response.json();",
              "  pm.expect(data).to.have.property('endpoints');",
              "  pm.expect(data.endpoints).to.include('/getAll');",
              "});"
            ]
          }
        }
      ]
    },
    {
      "name": "Get All Products",
      "request": {
        "method": "GET",
        "url": "{{baseUrl}}/getAll"
      },
      "event": [
        {
          "listen": "test",
          "script": {
            "exec": [
              "pm.test('Status 200', function ()
{ pm.response.to.have.status(200); });",
              "pm.test('Returns an array', function ()
{ pm.expect(pm.response.json()).to.be.an('array'); });"
            ]
          }
        }
      ]
    },
    {
      "name": "Get Single Product (1001)",
      "request": {
        "method": "GET",
        "url": "{{baseUrl}}/getSingleProduct/1001"
      },
      "event": [
        {
          "listen": "test",
          "script": {
            "exec": [
              "pm.test('Status 200', function ()
{ pm.response.to.have.status(200); });",
              "pm.test('ProductID is 1001', function ()
{ pm.expect(pm.response.json().ProductID).to.eql('1001'); });"
```

```
            ]
          }
        }
      ]
    },
    {
      "name": "Starts With (S)",
      "request": {
        "method": "GET",
        "url": "{{baseUrl}}/startsWith/S"
      },
      "event": [
        {
          "listen": "test",
          "script": {
            "exec": [
              "pm.test('Status 200', function ()
{ pm.response.to.have.status(200); });",
              "pm.test('Returns array', function ()
{ pm.expect(pm.response.json()).to.be.an('array'); });"
            ]
          }
        }
      ]
    },
    {
      "name": "Paginate (1001-1010)",
      "request": {
        "method": "GET",
        "url": "{{baseUrl}}/paginate/1001/1010"
      },
      "event": [
        {
          "listen": "test",
          "script": {
            "exec": [
              "pm.test('Status 200', function ()
{ pm.response.to.have.status(200); });",
              "pm.test('Returns array', function ()
{ pm.expect(pm.response.json()).to.be.an('array'); });"
            ]
          }
        }
      ]
    },
    {
      "name": "Convert (1001 USD->EUR)",
      "request": {
```

```
        "method": "GET",
        "url": "{{baseUrl}}/convert/1001"
      },
      "event": [
        {
          "listen": "test",
          "script": {
            "exec": [
              "pm.test('Status 200', function ()
{ pm.response.to.have.status(200); });",
              "pm.test('Has PriceEUR', function ()
{ pm.expect(pm.response.json()).to.have.property('PriceEUR'); });"
            ]
          }
        }
      ]
    },
    {
      "name": "Add New Product (9998)",
      "request": {
        "method": "POST",
        "url":
"{{baseUrl}}/addNew?ProductID=9998&Name=Test%20Product&UnitPrice=10.50&Stock
Quantity=5&Description=Test%20Description"
      },
      "event": [
        {
          "listen": "test",
          "script": {
            "exec": [
              "pm.test('Status 200', function ()
{ pm.response.to.have.status(200); });",
              "pm.test('Added successfully', function ()
{ pm.expect(pm.response.json().message).to.include('added'); });"
            ]
          }
        }
      ]
    },
    {
      "name": "Get Single Product (9998)",
      "request": {
        "method": "GET",
        "url": "{{baseUrl}}/getSingleProduct/9998"
      },
      "event": [
        {
          "listen": "test",
```

```
          "script": {
            "exec": [
              "pm.test('Status 200', function ()
{ pm.response.to.have.status(200); });",
              "pm.test('ProductID is 9998', function ()
{ pm.expect(pm.response.json().ProductID).to.eql('9998'); });"
            ]
          }
        }
      ]
    },
    {
      "name": "Delete Product (9998)",
      "request": {
        "method": "DELETE",
        "url": "{{baseUrl}}/deleteOne/9998"
      },
      "event": [
        {
          "listen": "test",
          "script": {
            "exec": [
              "pm.test('Status 200', function ()
{ pm.response.to.have.status(200); });",
              "pm.test('Deleted successfully', function ()
{ pm.expect(pm.response.json().message).to.include('deleted
successfully'); });"
            ]
          }
        }
      ]
    },
    {
      "name": "Get Deleted Product (9998) -> 404",
      "request": {
        "method": "GET",
        "url": "{{baseUrl}}/getSingleProduct/9998"
      },
      "event": [
        {
          "listen": "test",
          "script": {
            "exec": [
              "pm.test('Status 404', function ()
{ pm.response.to.have.status(404); });"
            ]
          }
        }
```

```
      ]
    }
  ]
}
```

Docker Configuration Files:

1. *Dockerfile*

```
# syntax=docker/dockerfile:1

FROM ubuntu:24.04

ENV DEBIAN_FRONTEND=noninteractive \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

RUN apt-get update \
  && apt-get install -y --no-install-recommends \
    ca-certificates \
    python3 \
    python3-pip \
    python3-venv \
  && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Ubuntu enables PEP 668 (externally managed env). Install deps into a venv.
RUN python3 -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

COPY requirements.txt ./
RUN pip install --no-cache-dir --upgrade pip \
  && pip install --no-cache-dir -r requirements.txt

COPY api.py ./
COPY convertToJSON.py ./
COPY dashboard.py ./
COPY seed_mongo.py ./
COPY tests ./tests
COPY data ./data

EXPOSE 8000

CMD ["python3", "-m", "uvicorn", "api:app", "--host", "0.0.0.0", "--port",
"8000"]
```

2. *Docker-compose.yaml*

```yaml
services:
  mongo:
    image: mongo:latest
    container_name: mongo-inventory
    restart: unless-stopped
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: adminPassword123
    volumes:
      - mongo_data:/data/db

  api:
    build: .
    container_name: api-inventory
    restart: unless-stopped
    depends_on:
      - mongo
    ports:
      - "8000:8000"
    environment:
      MONGO_URI:
mongodb://admin:adminPassword123@mongo:27017/?authSource=admin
      MONGO_DB: inventory_db
      EXCHANGE_RATE_API_KEY: ${EXCHANGE_RATE_API_KEY:-}

  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    restart: unless-stopped
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
    depends_on:
      - api

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    restart: unless-stopped
    ports:
```

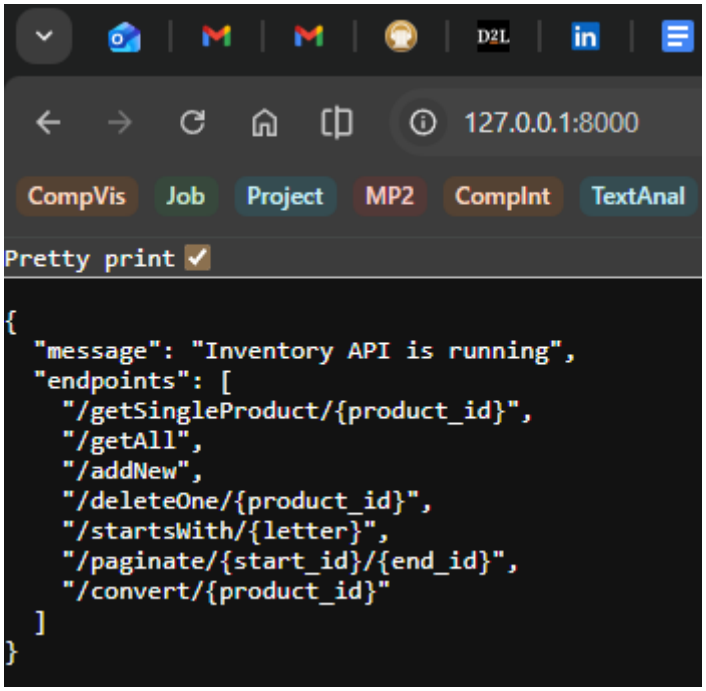
```yaml
      - "3000:3000"
    depends_on:
      - prometheus
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: admin
      GF_SECURITY_ALLOW_EMBEDDING: "true"
      GF_AUTH_ANONYMOUS_ENABLED: "true"
      GF_AUTH_ANONYMOUS_ORG_ROLE: Viewer
    volumes:
      - grafana_data:/var/lib/grafana
      - ./grafana/provisioning:/etc/grafana/provisioning

  dashboard:
    build: .
    container_name: dashboard
    restart: unless-stopped
    command: python3 -m uvicorn dashboard:app --host 0.0.0.0 --port 8001
    depends_on:
      - api
      - grafana
    ports:
      - "8001:8001"

  jenkins:
    build:
      context: ./jenkins
    image: webservassignment-jenkins:lts-docker
    container_name: jenkins
    restart: unless-stopped
    user: root
    ports:
      - "8081:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    environment:
      JAVA_OPTS: -Djenkins.install.runSetupWizard=false

volumes:
  mongo_data:
  jenkins_home:
  grafana_data:
```

3. *Jenkin- docker*

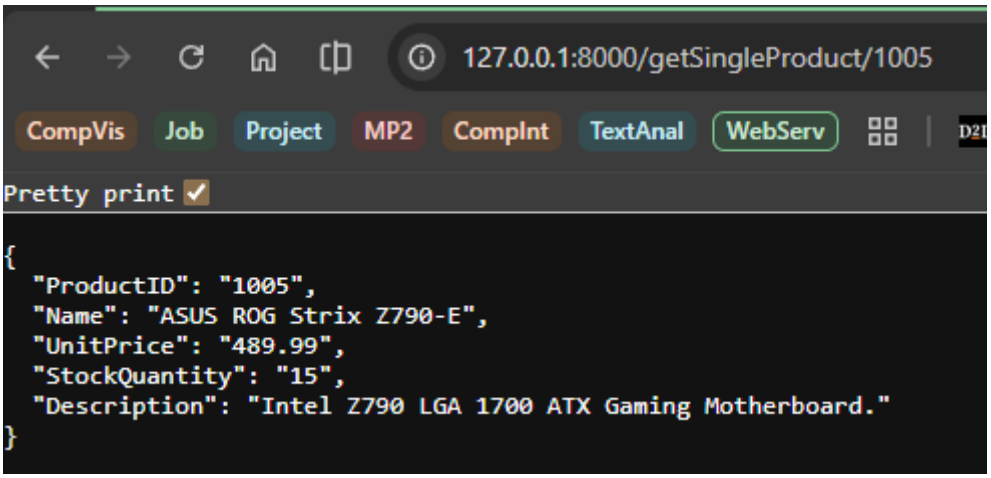
```
FROM jenkins/jenkins:lts-jdk17

USER root

# Install docker CLI inside the Jenkins container.
# Jenkins connects to the host Docker engine via /var/run/docker.sock
(mounted in docker-compose.yaml).
#
# Compose v2 plugin package availability varies by Debian release, so we
install the official
# CLI plugin binary to ensure `docker compose` works.
ARG DOCKER_COMPOSE_VERSION=2.39.1
RUN apt-get update \
  && apt-get install -y --no-install-recommends \
    ca-certificates \
    curl \
    git \
    docker-cli \
    docker.io \
  && rm -rf /var/lib/apt/lists/* \
  && mkdir -p /usr/local/lib/docker/cli-plugins \
  && curl -fsSL -o /usr/local/lib/docker/cli-plugins/docker-compose \
    "https://github.com/docker/compose/releases/download/v${DOCKER_COMPOSE_V
ERSION}/docker-compose-linux-x86_64" \
  && chmod +x /usr/local/lib/docker/cli-plugins/docker-compose \
  && docker compose version

USER jenkins
```

API endpoint Screenshots:

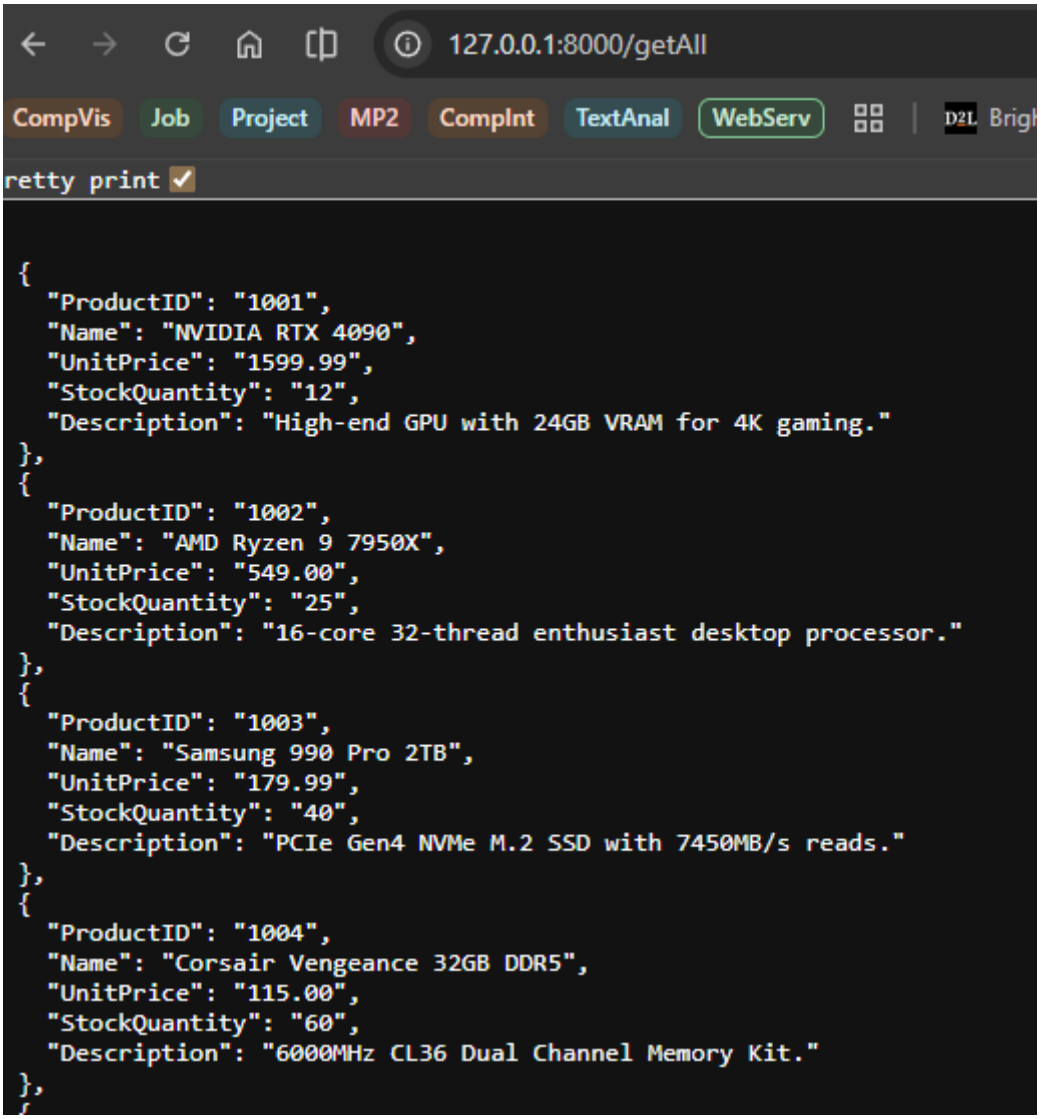
- **/** - Shows available api endpoints



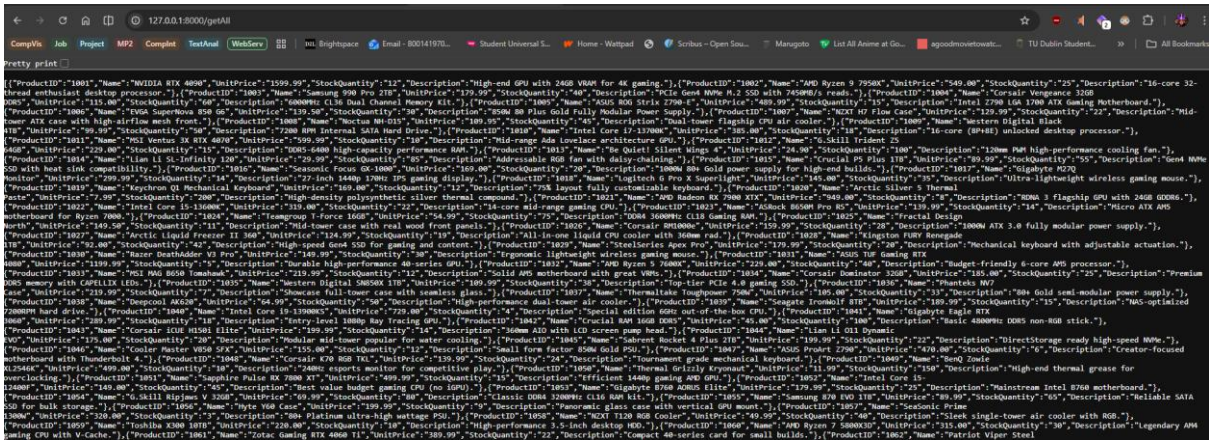
- **/getSingleProduct** - This allows you to pass a single ID number into the endpoint and return the details of the single product in JSON format.



- **/getAll** - This endpoint should return all inventory in JSON format from the database.

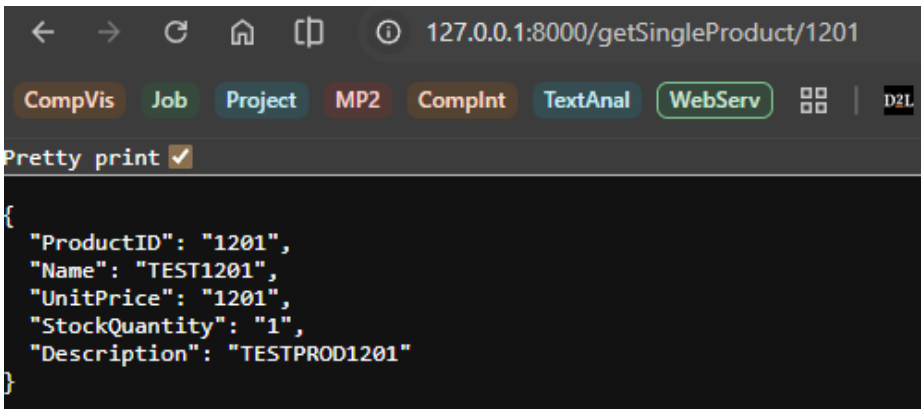



```
{
  "ProductID": "1001",
  "Name": "NVIDIA RTX 4090",
  "UnitPrice": "1599.99",
  "StockQuantity": "12",
  "Description": "High-end GPU with 24GB VRAM for 4K gaming."
},
{
  "ProductID": "1002",
  "Name": "AMD Ryzen 9 7950X",
  "UnitPrice": "549.00",
  "StockQuantity": "25",
  "Description": "16-core 32-thread enthusiast desktop processor."
},
{
  "ProductID": "1003",
  "Name": "Samsung 990 Pro 2TB",
  "UnitPrice": "179.99",
  "StockQuantity": "40",
  "Description": "PCIe Gen4 NVMe M.2 SSD with 7450MB/s reads."
},
{
  "ProductID": "1004",
  "Name": "Corsair Vengeance 32GB DDR5",
  "UnitPrice": "115.00",
  "StockQuantity": "60",
  "Description": "6000MHz CL36 Dual Channel Memory Kit."
},
```

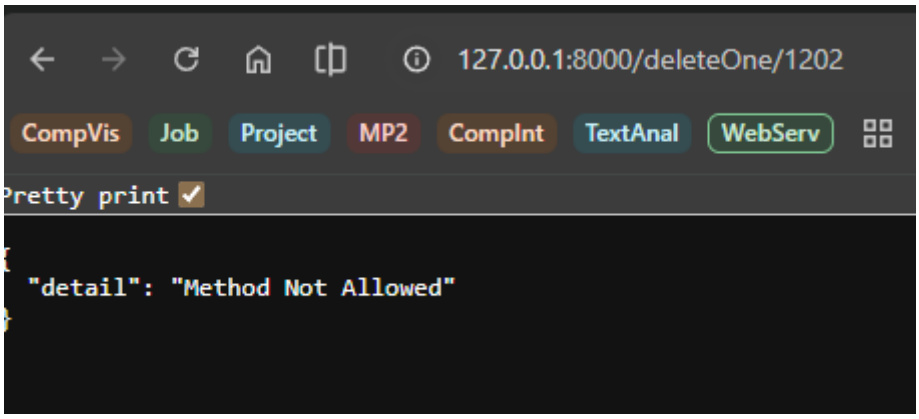



- **/addNew** - This endpoint should take in all 5 attributes of a new item and insert them into the database as a new record.

```
C:\Users\stanr\Desktop\Projects\TU-Dublin-Weekly\SEM02\WebServ\WebServAssignment-2026>curl -s "http://
127.0.0.1:8000/getSingleProduct/1005" | python -m json.tool
{
    "ProductID": "1005",
    "Name": "ASUS ROG Strix Z790-E",
    "UnitPrice": "489.99",
    "StockQuantity": "15",
    "Description": "Intel Z790 LGA 1700 ATX Gaming Motherboard."
}

C:\Users\stanr\Desktop\Projects\TU-Dublin-Weekly\SEM02\WebServ\WebServAssignment-2026>curl -X POST "ht
tp://127.0.0.1:8000/addNew?ProductID=1201&Name=TEST1201&UnitPrice=1201&StockQuantity=1&Description=TES
TPROD1201"
Internal Server Error
C:\Users\stanr\Desktop\Projects\TU-Dublin-Weekly\SEM02\WebServ\WebServAssignment-2026>curl -X POST "ht
tp://127.0.0.1:8000/addNew?ProductID=1201&Name=TEST1201&UnitPrice=1201&StockQuantity=1&Description=TES
TPROD1201"
{"detail":"ProductID already exists"}
C:\Users\stanr\Desktop\Projects\TU-Dublin-Weekly\SEM02\WebServ\WebServAssignment-2026>curl -s "http://
127.0.0.1:8000/getSingleProduct/1201" | python -m json.tool
{
    "ProductID": "1201",
    "Name": "TEST1201",
    "UnitPrice": "1201",
    "StockQuantity": "1",
    "Description": "TESTPROD1201"
}
```

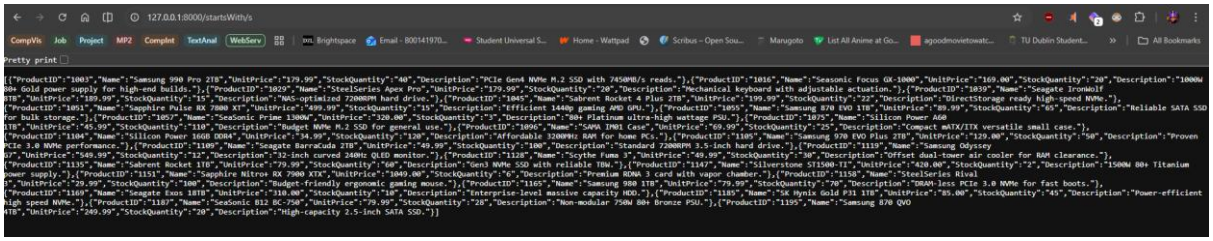


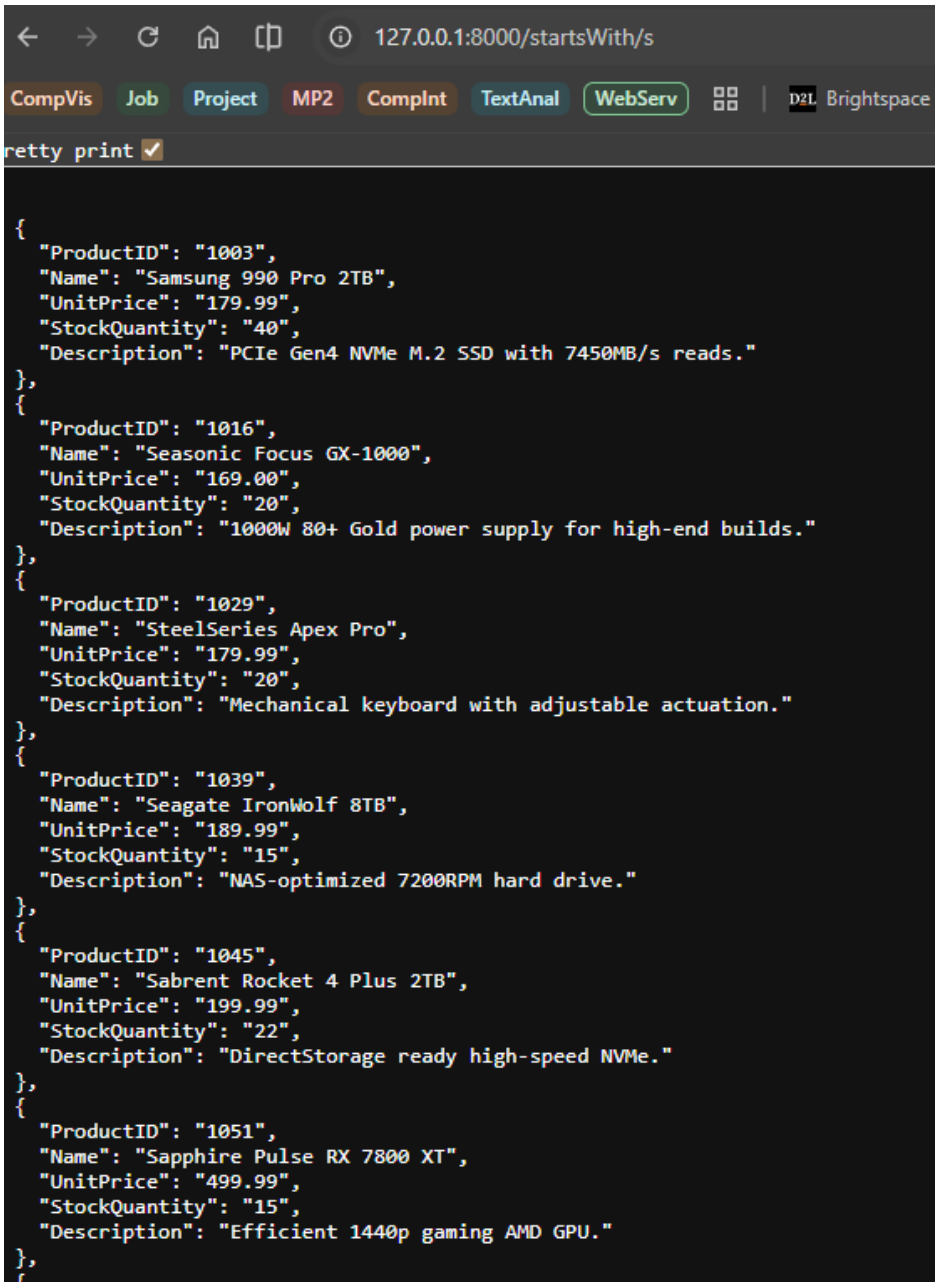
- **/deleteOne** - This endpoint should delete a product by the provided ID.




```
INFO:     172.18.0.1:54924 - "DELETE /deleteOne/1202 HTTP/1.1" 404 Not Found
INFO:     172.18.0.1:53668 - "GET /deleteOne/1202 HTTP/1.1" 405 Method Not Allowed
INFO:     172.18.0.1:55776 - "DELETE /deleteOne/1202 HTTP/1.1" 404 Not Found
INFO:     172.18.0.1:36726 - "DELETE /deleteOne/1201 HTTP/1.1" 200 OK
```

- **/startsWith** - This should allow the user to pass a letter to the URL, such as **s**, and return all products that start with s.






```
{
  "ProductID": "1003",
  "Name": "Samsung 990 Pro 2TB",
  "UnitPrice": "179.99",
  "StockQuantity": "40",
  "Description": "PCIe Gen4 NVMe M.2 SSD with 7450MB/s reads."
},
{
  "ProductID": "1016",
  "Name": "Seasonic Focus GX-1000",
  "UnitPrice": "169.00",
  "StockQuantity": "20",
  "Description": "1000W 80+ Gold power supply for high-end builds."
},
{
  "ProductID": "1029",
  "Name": "SteelSeries Apex Pro",
  "UnitPrice": "179.99",
  "StockQuantity": "20",
  "Description": "Mechanical keyboard with adjustable actuation."
},
{
  "ProductID": "1039",
  "Name": "Seagate IronWolf 8TB",
  "UnitPrice": "189.99",
  "StockQuantity": "15",
  "Description": "NAS-optimized 7200RPM hard drive."
},
{
  "ProductID": "1045",
  "Name": "Sabrent Rocket 4 Plus 2TB",
  "UnitPrice": "199.99",
  "StockQuantity": "22",
  "Description": "DirectStorage ready high-speed NVMe."
},
{
  "ProductID": "1051",
  "Name": "Sapphire Pulse RX 7800 XT",
  "UnitPrice": "499.99",
  "StockQuantity": "15",
  "Description": "Efficient 1440p gaming AMD GPU."
},
```


- **/paginate** - This URL should pass in a product ID to start from and a product ID to **end** from. The products should be returned in a batch of 10.

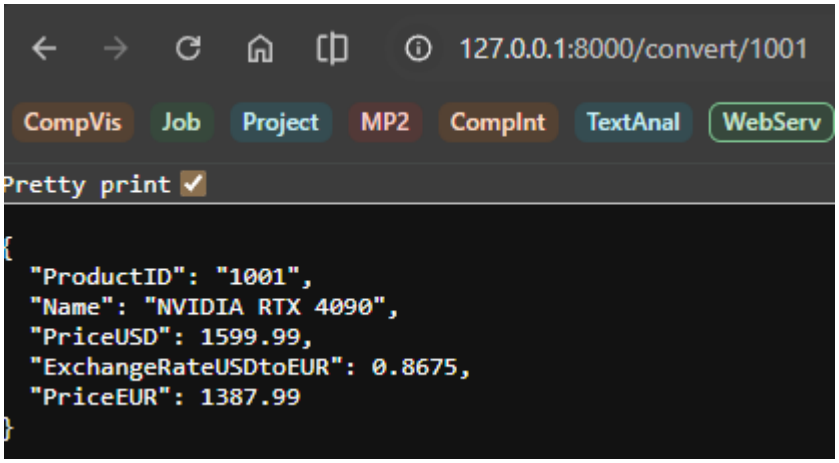
127.0.0.1:8000/paginate/1001/1005

CompVis Job Project MP2 CompInt TextAnal WebServ D2L Brights

Pretty print


```
[
  {
    "ProductID": "1001",
    "Name": "NVIDIA RTX 4090",
    "UnitPrice": "1599.99",
    "StockQuantity": "12",
    "Description": "High-end GPU with 24GB VRAM for 4K gaming."
  },
  {
    "ProductID": "1002",
    "Name": "AMD Ryzen 9 7950X",
    "UnitPrice": "549.00",
    "StockQuantity": "25",
    "Description": "16-core 32-thread enthusiast desktop processor."
  },
  {
    "ProductID": "1003",
    "Name": "Samsung 990 Pro 2TB",
    "UnitPrice": "179.99",
    "StockQuantity": "40",
    "Description": "PCIe Gen4 NVMe M.2 SSD with 7450MB/s reads."
  },
  {
    "ProductID": "1004",
    "Name": "Corsair Vengeance 32GB DDR5",
    "UnitPrice": "115.00",
    "StockQuantity": "60",
    "Description": "6000MHz CL36 Dual Channel Memory Kit."
  },
  {
    "ProductID": "1005",
    "Name": "ASUS ROG Strix Z790-E",
    "UnitPrice": "489.99",
    "StockQuantity": "15",
    "Description": "Intel Z790 LGA 1700 ATX Gaming Motherboard."
  }
```


- **/convert** - All of the prices are currently in dollars in the sample data. Implement a URL titled **/convert** which takes in the ID number of a product and returns the price in euros. An online API should be used to get the current exchange rate.



Jenkins:

1. *Git Pull*

Console Output

```
The recommended git tool is: NONE
No credentials specified
 > git rev-parse --resolve-git-dir /var/jenkins_home/workspace/Assignment_2026_Pipeline/.git # timeout=10
Fetching changes from the remote Git repository
 > git config remote.origin.url https://github.com/Stan-T-Pet/WebServAssignment-2026.git # timeout=10
Fetching upstream changes from https://github.com/Stan-T-Pet/WebServAssignment-2026.git
 > git --version # timeout=10
 > git --version # 'git version 2.47.3'
 > git fetch --tags --force --progress -- https://github.com/Stan-T-Pet/WebServAssignment-2026.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 2e81f62825ee1187874538acbfe7e35f6f72ca0b (refs/remotes/origin/main)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 2e81f62825ee1187874538acbfe7e35f6f72ca0b # timeout=10
 > git branch -a -v --no-abbrev # timeout=10
 > git branch -D main # timeout=10
 > git checkout -b main 2e81f62825ee1187874538acbfe7e35f6f72ca0b # timeout=10
Commit message: "Refactor README generation in Jenkinsfile to use a dedicated script and improve readability"
 > git rev-list --no-walk 2e81f62825ee1187874538acbfe7e35f6f72ca0b # timeout=10
```

```
stages {
  stage('Pull From GitHub') {
    steps {
      git branch: 'main', url: 'https://github.com/Stan-T-Pet/WebServAssignment-2026.git'
    }
  }
```

2. Docker Ubuntu Container

Console Output

```
+ set -eux
+ NETWORK=ci-11
+ MONGO_NAME=mongo-ci-11
+ API_NAME=api-ci-11
+ API_IMAGE=api-ubuntu:11
+ docker network rm ci-11
+ true
+ docker network create ci-11
569202ff16f298bc221329e7668c8d8882109cbd7b40f7f6b13a671e5ada447c
+ docker rm -f mongo-ci-11
+ docker run -d --name mongo-ci-11 --network ci-11 -e MONGO_INITDB_ROOT_USERNAME=admin -e MONGO_INITDB_ROOT_PASSWORD=adminPassword123 mongo:latest
8174ea352894dd72ce583ebe836bd5433abc9a839e88dd9c59e621c55f1a4749
+ docker build -t api-ubuntu:11 -f Dockerfile .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  579.1kB
```

```
Sending build context to Docker daemon  579.1kB

Step 1/16 : FROM ubuntu:24.04
 ---> 186072bba1b2
Step 2/16 : ENV DEBIAN_FRONTEND=noninteractive     PYTHONDONTWRITEBYTECODE=1     PYTHONUNBUFFERED=1
 ---> Using cache
 ---> 65d990a6c0a1
Step 3/16 : RUN apt-get update   && apt-get install -y --no-install-recommends     ca-certificates     python3     python3-pip     python3-venv   && rm -rf /var/lib/apt/lists/*
 ---> Using cache
 ---> ba093c943ae1
Step 4/16 : WORKDIR /app
 ---> Using cache
 ---> 1f8c07690285
Step 5/16 : RUN python3 -m venv /opt/venv
 ---> Using cache
 ---> 21462b25f8e0
Step 6/16 : ENV PATH="/opt/venv/bin:$PATH"
 ---> Using cache
 ---> 28f599cc8875
Step 7/16 : COPY requirements.txt ./
 ---> Using cache
 ---> 7cecf79ee75e
Step 8/16 : RUN pip install --no-cache-dir --upgrade pip   && pip install --no-cache-dir -r requirements.txt
 ---> Using cache
 ---> eb9f6c90ee8e
Step 9/16 : COPY api.py ./
 ---> Using cache
 ---> 97cdbdb285ee
Step 10/16 : COPY convertToJSON.py ./
 ---> Using cache
 ---> caba4e10e492
Step 11/16 : COPY dashboard.py ./
 ---> Using cache
 ---> f937dbc6ceba
Step 12/16 : COPY seed_mongo.py ./
 ---> Using cache
 ---> bfd1285949c5
Step 13/16 : COPY tests ./tests
 ---> Using cache
 ---> 02bf151b5d2c
```

```
 ---> bfd1285949c5
Step 13/16 : COPY tests ./tests
 ---> Using cache
 ---> 02bf151b5d2c
Step 14/16 : COPY data ./data
 ---> Using cache
 ---> 10e9a6f65d4c
Step 15/16 : EXPOSE 8000
 ---> Using cache
 ---> f3719288cb63
Step 16/16 : CMD ["python3", "-m", "uvicorn", "api:app", "--host", "0.0.0.0", "--port", "8000"]
 ---> Using cache
 ---> 01e394305e67
Successfully built 01e394305e67
Successfully tagged api-ubuntu:11
+ docker run --rm api-ubuntu:11 python3 -m pytest -q
...........                                                              [100%]
11 passed in 0.44s
+ seq 1 20
+ docker run --rm --network ci-11 -e MONGO_URI=mongodb://admin:adminPassword123@mongo-ci-11:27017/?authSource=admin -e MONGO_DB=inventory_db api-ubuntu:11 python3 seed_mongo.py
MongoDB seeded successfully.
+ echo MongoDB seeded
MongoDB seeded
+ break
+ docker rm -f api-ci-11
+ docker run -d --name api-ci-11 --network ci-11 -e MONGO_URI=mongodb://admin:adminPassword123@mongo-ci-11:27017/?authSource=admin -e MONGO_DB=inventory_db -e EXCHANGE_RATE_API_KEY= api-ubuntu:11
d65728ae0f381e00b477d8dbce9f772419cb3048a00a6fff4a076efdd1a5a90
+ seq 1 60
+ docker run --rm --network ci-11 curlimages/curl:8.7.1 -fsS http://api-ci-11:8000/
curl: (7) Failed to connect to api-ci-11 port 8000 after 0 ms: Couldn't connect to server
+ sleep 2
+ docker run --rm --network ci-11 curlimages/curl:8.7.1 -fsS http://api-ci-11:8000/
+ echo API is up
API is up
+ exit 0
```

```
stage('Build and Run Ubuntu Container in Background') {
    steps {
      sh '''
        set -eux

        NETWORK="ci-${BUILD_NUMBER}"
        MONGO_NAME="mongo-ci-${BUILD_NUMBER}"
        API_NAME="api-ci-${BUILD_NUMBER}"
        API_IMAGE="api-ubuntu:${BUILD_NUMBER}"

        # Fresh network for this build
        docker network rm "$NETWORK" >/dev/null 2>&1 || true
        docker network create "$NETWORK"

        # Start MongoDB (background)
        docker rm -f "$MONGO_NAME" >/dev/null 2>&1 || true
        docker run -d --name "$MONGO_NAME" --network "$NETWORK" \
          -e MONGO_INITDB_ROOT_USERNAME=admin \
          -e MONGO_INITDB_ROOT_PASSWORD=adminPassword123 \
          mongo:latest

        # Build an Ubuntu-based image for the API (contains the code pulled from
GitHub)
        docker build -t "$API_IMAGE" -f Dockerfile .

        # Run Python unit tests inside the Ubuntu container image
        docker run --rm "$API_IMAGE" python3 -m pytest -q

        # Seed Mongo with products.json
        for i in $(seq 1 20); do
          if docker run --rm --network "$NETWORK" \
```

```
          -e
MONGO_URI="mongodb://admin:adminPassword123@${MONGO_NAME}:27017/?a
uthSource=admin" \
          -e MONGO_DB="inventory_db" \
          "$API_IMAGE" \
          python3 seed_mongo.py; then
          echo "MongoDB seeded"
          break
        fi
        sleep 2
      done

      # Start API (background)
      docker rm -f "$API_NAME" >/dev/null 2>&1 || true
      docker run -d --name "$API_NAME" --network "$NETWORK" \
        -e
MONGO_URI="mongodb://admin:adminPassword123@${MONGO_NAME}:27017/?a
uthSource=admin" \
        -e MONGO_DB="inventory_db" \
        -e EXCHANGE_RATE_API_KEY="${EXCHANGE_RATE_API_KEY:-}" \
        "$API_IMAGE"

      # Wait until the API is reachable
      for i in $(seq 1 60); do
        if docker run --rm --network "$NETWORK" curlimages/curl:8.7.1 -fsS
"http://${API_NAME}:8000/" >/dev/null; then
          echo "API is up"
          exit 0
        fi
        sleep 2
      done

      echo "API did not become ready in time"
      docker logs "$API_NAME" || true
      exit 1
    '''
  }
}
```

3. Newman Outputs

Console Output

```
+ set -eux
+ NETWORK=ci-11
+ API_NAME=api-ci-11
+ NEWMAN_NAME=newman-ci-11
+ docker rm -f newman-ci-11
+ docker create --name newman-ci-11 --network ci-11 postman/newman:alpine run /etc/newman/collection.json --env-var baseUrl=http://api-ci-11:8000 -r cli,junit --reporter-junit-export /etc/newman/newman-report.xml
2d6a3bfeffbbb89f105a2fbdc3c8c848ad3937f10ebf2069836a28ba9975543b
+ docker cp postman/InventoryAPI.postman_collection.json newman-ci-11:/etc/newman/collection.json
+ docker start -a newman-ci-11
newman

Inventory API - Endpoint Tests

→ Home
  GET http://api-ci-11:8000/ [200 OK, 336B, 49ms]
  ✓  Status 200
  ✓  Has endpoints list

→ Get All Products
  GET http://api-ci-11:8000/getAll [200 OK, 29.56kB, 13ms]
  ✓  Status 200
  ✓  Returns an array

→ Get Single Product (1001)
  GET http://api-ci-11:8000/getSingleProduct/1001 [200 OK, 273B, 4ms]
  ✓  Status 200
  ✓  ProductID is 1001

→ Starts With (S)
  GET http://api-ci-11:8000/startsWith/S [200 OK, 3.64kB, 3ms]
  ✓  Status 200
  ✓  Returns array

→ Paginate (1001-1010)
  GET http://api-ci-11:8000/paginate/1001/1010 [200 OK, 1.64kB, 3ms]
  ✓  Status 200
  ✓  Returns array
```

```
→ Paginate (1001-1010)
  GET http://api-ci-11:8000/paginate/1001/1010 [200 OK, 1.64kB, 3ms]
  ✓  Status 200
  ✓  Returns array

→ Convert (1001 USD->EUR)
  GET http://api-ci-11:8000/convert/1001 [200 OK, 239B, 61ms]
  ✓  Status 200
  ✓  Has PriceEUR

→ Add New Product (9998)
  POST http://api-ci-11:8000/addNew?ProductID=9998&Name=Test%20Product&UnitPrice=10.50&StockQuantity=5&Description=Test%20Description [200 OK, 291B, 4ms]
  ✓  Status 200
  ✓  Added successfully

→ Get Single Product (9998)
  GET http://api-ci-11:8000/getSingleProduct/9998 [200 OK, 240B, 3ms]
  ✓  Status 200
  ✓  ProductID is 9998

→ Delete Product (9998)
  DELETE http://api-ci-11:8000/deleteOne/9998 [200 OK, 172B, 3ms]
  ✓  Status 200
  ✓  Deleted successfully

→ Get Deleted Product (9998) -> 404
  GET http://api-ci-11:8000/getSingleProduct/9998 [404 Not Found, 162B, 3ms]
  ✓  Status 404
```

```
┌─────────────────────────┬──────────────────┬──────────────────┐
│                         │         executed │           failed │
├─────────────────────────┼──────────────────┼──────────────────┤
│              iterations │                1 │                0 │
├─────────────────────────┼──────────────────┼──────────────────┤
│                requests │               10 │                0 │
├─────────────────────────┼──────────────────┼──────────────────┤
│            test-scripts │               10 │                0 │
├─────────────────────────┼──────────────────┼──────────────────┤
│      prerequest-scripts │                0 │                0 │
├─────────────────────────┼──────────────────┼──────────────────┤
│              assertions │               19 │                0 │
├─────────────────────────┴──────────────────┴──────────────────┤
│ total run duration: 301ms                                      │
├────────────────────────────────────────────────────────────────┤
│ total data received: 35.29kB (approx)                          │
├────────────────────────────────────────────────────────────────┤
│ average response time: 14ms [min: 3ms, max: 61ms, s.d.: 20ms] │
└────────────────────────────────────────────────────────────────┘
+ docker cp newman-ci-11:/etc/newman/newman-report.xml newman-report.xml
+ docker rm -f newman-ci-11
```

```
stage('Run Postman Tests (Newman)') {
    steps {
      sh '''
        set -eux

        NETWORK="ci-${BUILD_NUMBER}"
        API_NAME="api-ci-${BUILD_NUMBER}"
        NEWMAN_NAME="newman-ci-${BUILD_NUMBER}"

        docker rm -f "$NEWMAN_NAME" >/dev/null 2>&1 || true

        # Avoid mounting $WORKSPACE (paths are interpreted on the Docker host). Use
docker cp instead.
        docker create --name "$NEWMAN_NAME" --network "$NETWORK"
postman/newman:alpine \
          run /etc/newman/collection.json \
          --env-var baseUrl="http://${API_NAME}:8000" \
          -r cli,junit \
          --reporter-junit-export /etc/newman/newman-report.xml

        docker cp postman/InventoryAPI.postman_collection.json
"$NEWMAN_NAME:/etc/newman/collection.json"
```

```
        docker start -a "$NEWMAN_NAME"
        docker cp "$NEWMAN_NAME:/etc/newman/newman-report.xml" newman-report.xml
        docker rm -f "$NEWMAN_NAME" >/dev/null 2>&1 || true
      '''
    }
  }
```

4. *Kill Docker*

Console Output

```
+ set -eux
+ NETWORK=ci-11
+ API_NAME=api-ci-11
+ API_IMAGE=api-ubuntu:11
+ README_NAME=readme-ci-11
+ ZIP_NAME=zip-ci-11
+ cat
+ cat
+ docker rm -f readme-ci-11
+ docker create --name readme-ci-11 --network ci-11 -e API_BASE_URL=http://api-ci-11:8000 api-ubuntu:11 python3 /tmp/readme_gen.py
1fdd8ea0e8f8ef2e150262084e424964251bd68a29ead4b14b6d7aefa7354c27
+ docker cp /tmp/readme_gen.py readme-ci-11:/tmp/readme_gen.py
+ docker start -a readme-ci-11
+ docker rm -f readme-ci-11
+ docker rm -f zip-ci-11
+ docker create --name zip-ci-11 -w /work python:3.12-slim python make_zip.py
dd173637d4d15dee8e95bd793de6fe83fc5fe78662a39c6a9bf284b938584642
+ docker cp api.py zip-ci-11:/work/api.py
+ docker cp convertToJSON.py zip-ci-11:/work/convertToJSON.py
+ docker cp dashboard.py zip-ci-11:/work/dashboard.py
+ docker cp seed_mongo.py zip-ci-11:/work/seed_mongo.py
+ docker cp requirements.txt zip-ci-11:/work/requirements.txt
+ docker cp Dockerfile zip-ci-11:/work/Dockerfile
+ docker cp docker-compose.yaml zip-ci-11:/work/docker-compose.yaml
+ docker cp Jenkinsfile zip-ci-11:/work/Jenkinsfile
+ docker cp README.txt zip-ci-11:/work/README.txt
+ docker cp postman zip-ci-11:/work/postman
+ docker cp data zip-ci-11:/work/data
+ docker cp prometheus.yml zip-ci-11:/work/prometheus.yml
+ docker cp grafana zip-ci-11:/work/grafana
+ docker cp /tmp/make_zip.py zip-ci-11:/work/make_zip.py
+ docker start -a zip-ci-11
artifacts/complete-20260405-223907.zip
+ rm -rf artifacts
+ docker cp zip-ci-11:/work/artifacts artifacts
+ docker rm -f zip-ci-11
```

```
stage('Generate README.txt and Zip Artifact') {
  steps {
    sh '''
      set -eux

      NETWORK="ci-${BUILD_NUMBER}"
      API_NAME="api-ci-${BUILD_NUMBER}"
      API_IMAGE="api-ubuntu:${BUILD_NUMBER}"
      README_NAME="readme-ci-${BUILD_NUMBER}"
      ZIP_NAME="zip-ci-${BUILD_NUMBER}"
```

```
        # Write README generator script to a temp file (heredoc terminator must be at
column 1)
        cat > /tmp/readme_gen.py <<'README_EOF'
import os
import json
import urllib.request

base = os.environ["API_BASE_URL"].rstrip("/")
openapi_url = base + "/openapi.json"
docs_url = base + "/docs"

with urllib.request.urlopen(openapi_url, timeout=20) as resp:
  spec = json.loads(resp.read().decode("utf-8"))

lines = []
lines.append("Inventory API - Endpoints\\n")
lines.append(f"OpenAPI JSON: {openapi_url}\\n")
lines.append(f"Swagger UI (FastAPI): {docs_url}\\n")
lines.append("FastAPI documentation: https://fastapi.tiangolo.com/\\n\\n")

paths = spec.get("paths", {})
for path, methods in sorted(paths.items()):
  for method, details in sorted(methods.items()):
    params = details.get("parameters", [])
    param_bits = []
    for p in params:
      name = p.get("name")
      where = p.get("in")
      required = p.get("required", False)
      schema = p.get("schema", {})
      ptype = schema.get("type", "")
      param_bits.append(
        f"{name} ({where}{', required' if required else ''}{', ' + ptype if ptype else ''})"
      )
    if param_bits:
      lines.append(f"{method.upper()} {path} - params: " + ", ".join(param_bits) + "\\n")
    else:
      lines.append(f"{method.upper()} {path}\\n")

print("".join(lines), end="")
README_EOF

        # Write the zip script to a temp file
        cat > /tmp/make_zip.py <<'ZIP_EOF'
from __future__ import annotations
import os
from datetime import datetime
```

```
from pathlib import Path
from zipfile import ZipFile, ZIP_DEFLATED

ts = datetime.now().strftime("%Y%m%d-%H%M%S")
out_dir = Path("artifacts")
out_dir.mkdir(exist_ok=True)

zip_name = out_dir / f"complete-{ts}.zip"

include_paths = [
  Path("api.py"),
  Path("convertToJSON.py"),
  Path("dashboard.py"),
  Path("seed_mongo.py"),
  Path("requirements.txt"),
  Path("Dockerfile"),
  Path("docker-compose.yaml"),
  Path("Jenkinsfile"),
  Path("README.txt"),
  Path("postman"),
  Path("data"),
  Path("prometheus.yml"),
  Path("grafana"),
]

def add_path(zipf: ZipFile, p: Path) -> None:
  if p.is_dir():
    for sub in sorted(p.rglob("*")):
      if sub.is_file():
        zipf.write(sub, sub.as_posix())
  elif p.is_file():
    zipf.write(p, p.as_posix())

with ZipFile(zip_name, "w", compression=ZIP_DEFLATED) as zipf:
  for p in include_paths:
    if p.exists():
      add_path(zipf, p)

print(zip_name.as_posix())
ZIP_EOF

      # Generate README.txt without bind mounts; Jenkins may be running in a
container.
      docker rm -f "$README_NAME" >/dev/null 2>&1 || true
      docker create --name "$README_NAME" --network "$NETWORK" \
       -e API_BASE_URL="http://${API_NAME}:8000" \
       "$API_IMAGE" \
```

```
        python3 /tmp/readme_gen.py
      docker cp /tmp/readme_gen.py "$README_NAME:/tmp/readme_gen.py"
      docker start -a "$README_NAME" > README.txt
      docker rm -f "$README_NAME" >/dev/null 2>&1 || true

      # Create the zip container, copy all files in, run it, extract artifacts
      docker rm -f "$ZIP_NAME" >/dev/null 2>&1 || true
      docker create --name "$ZIP_NAME" -w /work python:3.12-slim python
make_zip.py

      docker cp api.py "$ZIP_NAME:/work/api.py"
      docker cp convertToJSON.py "$ZIP_NAME:/work/convertToJSON.py"
      docker cp dashboard.py "$ZIP_NAME:/work/dashboard.py"
      docker cp seed_mongo.py "$ZIP_NAME:/work/seed_mongo.py"
      docker cp requirements.txt "$ZIP_NAME:/work/requirements.txt"
      docker cp Dockerfile "$ZIP_NAME:/work/Dockerfile"
      docker cp docker-compose.yaml "$ZIP_NAME:/work/docker-compose.yaml"
      docker cp Jenkinsfile "$ZIP_NAME:/work/Jenkinsfile"
      docker cp README.txt "$ZIP_NAME:/work/README.txt"
      docker cp postman "$ZIP_NAME:/work/postman"
      docker cp data "$ZIP_NAME:/work/data"
      docker cp prometheus.yml "$ZIP_NAME:/work/prometheus.yml"
      docker cp grafana "$ZIP_NAME:/work/grafana"
      docker cp /tmp/make_zip.py "$ZIP_NAME:/work/make_zip.py"

      docker start -a "$ZIP_NAME"
      rm -rf artifacts
      docker cp "$ZIP_NAME:/work/artifacts" artifacts
      docker rm -f "$ZIP_NAME" >/dev/null 2>&1 || true
    '''
   }
  }
 }
```

5. *Cleanup*

#11 / Shell Script / Console Output

Console Output

```
+ set +e
+ NETWORK=ci-11
+ MONGO_NAME=mongo-ci-11
+ API_NAME=api-ci-11
+ docker rm -f api-ci-11
+ docker rm -f mongo-ci-11
+ docker network rm ci-11
```

```
post {
 always {
  sh '''
   set +e
   NETWORK="ci-${BUILD_NUMBER}"
   MONGO_NAME="mongo-ci-${BUILD_NUMBER}"
   API_NAME="api-ci-${BUILD_NUMBER}"

   docker rm -f "$API_NAME" >/dev/null 2>&1 || true
   docker rm -f "$MONGO_NAME" >/dev/null 2>&1 || true
   docker network rm "$NETWORK" >/dev/null 2>&1 || true
  '''

  // Make outputs easy to download from Jenkins
  archiveArtifacts artifacts: 'README.txt,newman-report.xml,artifacts/complete-
*.zip', fingerprint: true, allowEmptyArchive: true

  echo 'Pipeline complete.'
 }
}
```

6. *Jenkins pipeline Full*

```
pipeline {
  agent any

  stages {
    stage('Pull From GitHub') {
```

```
      steps {
        git branch: 'main', url: 'https://github.com/Stan-T-
Pet/WebServAssignment-2026.git'
      }
    }

    stage('Build and Run Ubuntu Container in Background') {
      steps {
        sh '''
          set -eux

          NETWORK="ci-${BUILD_NUMBER}"
          MONGO_NAME="mongo-ci-${BUILD_NUMBER}"
          API_NAME="api-ci-${BUILD_NUMBER}"
          API_IMAGE="api-ubuntu:${BUILD_NUMBER}"

          # Fresh network for this build
          docker network rm "$NETWORK" >/dev/null 2>&1 || true
          docker network create "$NETWORK"

          # Start MongoDB (background)
          docker rm -f "$MONGO_NAME" >/dev/null 2>&1 || true
          docker run -d --name "$MONGO_NAME" --network "$NETWORK" \
            -e MONGO_INITDB_ROOT_USERNAME=admin \
            -e MONGO_INITDB_ROOT_PASSWORD=adminPassword123 \
            mongo:latest

          # Build an Ubuntu-based image for the API (contains the code
pulled from GitHub)
          docker build -t "$API_IMAGE" -f Dockerfile .

          # Run Python unit tests inside the Ubuntu container image
          docker run --rm "$API_IMAGE" python3 -m pytest -q

          # Seed Mongo with products.json
          for i in $(seq 1 20); do
            if docker run --rm --network "$NETWORK" \
              -e
MONGO_URI="mongodb://admin:adminPassword123@${MONGO_NAME}:27017/?authSource=
admin" \
              -e MONGO_DB="inventory_db" \
              "$API_IMAGE" \
              python3 seed_mongo.py; then
              echo "MongoDB seeded"
              break
            fi
            sleep 2
          done
```

```
          # Start API (background)
          docker rm -f "$API_NAME" >/dev/null 2>&1 || true
          docker run -d --name "$API_NAME" --network "$NETWORK" \
            -e
MONGO_URI="mongodb://admin:adminPassword123@${MONGO_NAME}:27017/?authSource=
admin" \
            -e MONGO_DB="inventory_db" \
            -e EXCHANGE_RATE_API_KEY="${EXCHANGE_RATE_API_KEY:-}" \
            "$API_IMAGE"

          # Wait until the API is reachable
          for i in $(seq 1 60); do
            if docker run --rm --network "$NETWORK" curlimages/curl:8.7.1 -
fsS "http://${API_NAME}:8000/" >/dev/null; then
              echo "API is up"
              exit 0
            fi
            sleep 2
          done

          echo "API did not become ready in time"
          docker logs "$API_NAME" || true
          exit 1
        '''
      }
    }

    stage('Run Postman Tests (Newman)') {
      steps {
        sh '''
          set -eux

          NETWORK="ci-${BUILD_NUMBER}"
          API_NAME="api-ci-${BUILD_NUMBER}"
          NEWMAN_NAME="newman-ci-${BUILD_NUMBER}"

          docker rm -f "$NEWMAN_NAME" >/dev/null 2>&1 || true

          # Avoid mounting $WORKSPACE (paths are interpreted on the Docker
host). Use docker cp instead.
          docker create --name "$NEWMAN_NAME" --network "$NETWORK"
postman/newman:alpine \
            run /etc/newman/collection.json \
            --env-var baseUrl="http://${API_NAME}:8000" \
            -r cli,junit \
            --reporter-junit-export /etc/newman/newman-report.xml
```

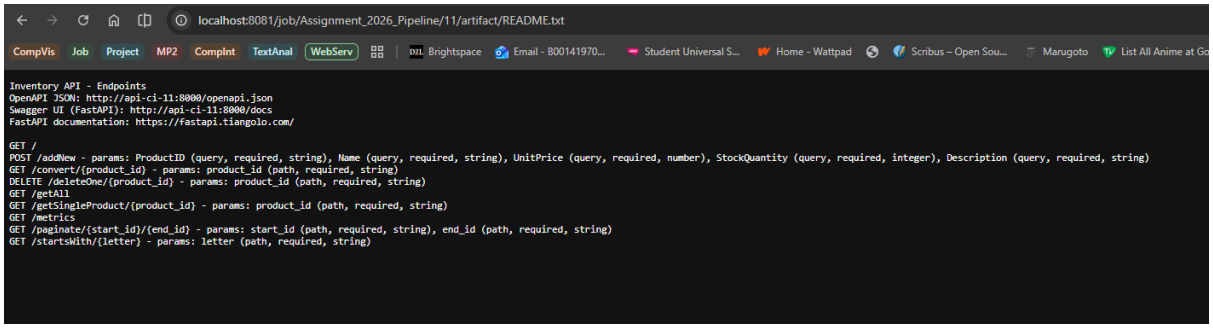
```
          docker cp postman/InventoryAPI.postman_collection.json
"$NEWMAN_NAME:/etc/newman/collection.json"
          docker start -a "$NEWMAN_NAME"
          docker cp "$NEWMAN_NAME:/etc/newman/newman-report.xml" newman-
report.xml
          docker rm -f "$NEWMAN_NAME" >/dev/null 2>&1 || true
        '''
      }
    }

    stage('Generate README.txt and Zip Artifact') {
      steps {
        sh '''
          set -eux

          NETWORK="ci-${BUILD_NUMBER}"
          API_NAME="api-ci-${BUILD_NUMBER}"
          API_IMAGE="api-ubuntu:${BUILD_NUMBER}"
          README_NAME="readme-ci-${BUILD_NUMBER}"
          ZIP_NAME="zip-ci-${BUILD_NUMBER}"

          # Write README generator script to a temp file (heredoc terminator
must be at column 1)
          cat > /tmp/readme_gen.py <<'README_EOF'
import os
import json
import urllib.request

base = os.environ["API_BASE_URL"].rstrip("/")
openapi_url = base + "/openapi.json"
docs_url = base + "/docs"

with urllib.request.urlopen(openapi_url, timeout=20) as resp:
  spec = json.loads(resp.read().decode("utf-8"))

lines = []
lines.append("Inventory API - Endpoints\\n")
lines.append(f"OpenAPI JSON: {openapi_url}\\n")
lines.append(f"Swagger UI (FastAPI): {docs_url}\\n")
lines.append("FastAPI documentation: https://fastapi.tiangolo.com/\\n\\n")

paths = spec.get("paths", {})
for path, methods in sorted(paths.items()):
  for method, details in sorted(methods.items()):
    params = details.get("parameters", [])
    param_bits = []
    for p in params:
      name = p.get("name")
```

```
      where = p.get("in")
      required = p.get("required", False)
      schema = p.get("schema", {})
      ptype = schema.get("type", "")
      param_bits.append(
        f"{name} ({where}{', required' if required else ''}{', ' + ptype if
ptype else ''})"
      )
    if param_bits:
      lines.append(f"{method.upper()} {path} - params: " + ",
".join(param_bits) + "\\n")
    else:
      lines.append(f"{method.upper()} {path}\\n")

print("".join(lines), end="")
README_EOF

          # Write the zip script to a temp file
          cat > /tmp/make_zip.py <<'ZIP_EOF'
from __future__ import annotations
import os
from datetime import datetime
from pathlib import Path
from zipfile import ZipFile, ZIP_DEFLATED

ts = datetime.now().strftime("%Y%m%d-%H%M%S")
out_dir = Path("artifacts")
out_dir.mkdir(exist_ok=True)

zip_name = out_dir / f"complete-{ts}.zip"

include_paths = [
    Path("api.py"),
    Path("convertToJSON.py"),
    Path("dashboard.py"),
    Path("seed_mongo.py"),
    Path("requirements.txt"),
    Path("Dockerfile"),
    Path("docker-compose.yaml"),
    Path("Jenkinsfile"),
    Path("README.txt"),
    Path("postman"),
    Path("data"),
    Path("prometheus.yml"),
    Path("grafana"),
]

def add_path(zipf: ZipFile, p: Path) -> None:
```

```
    if p.is_dir():
        for sub in sorted(p.rglob("*")):
            if sub.is_file():
                zipf.write(sub, sub.as_posix())
    elif p.is_file():
        zipf.write(p, p.as_posix())

with ZipFile(zip_name, "w", compression=ZIP_DEFLATED) as zipf:
    for p in include_paths:
        if p.exists():
            add_path(zipf, p)

print(zip_name.as_posix())
ZIP_EOF

          # Generate README.txt without bind mounts; Jenkins may be running
in a container.
          docker rm -f "$README_NAME" >/dev/null 2>&1 || true
          docker create --name "$README_NAME" --network "$NETWORK" \
            -e API_BASE_URL="http://${API_NAME}:8000" \
            "$API_IMAGE" \
            python3 /tmp/readme_gen.py
          docker cp /tmp/readme_gen.py "$README_NAME:/tmp/readme_gen.py"
          docker start -a "$README_NAME" > README.txt
          docker rm -f "$README_NAME" >/dev/null 2>&1 || true

          # Create the zip container, copy all files in, run it, extract
artifacts
          docker rm -f "$ZIP_NAME" >/dev/null 2>&1 || true
          docker create --name "$ZIP_NAME" -w /work python:3.12-slim python
make_zip.py

          docker cp api.py "$ZIP_NAME:/work/api.py"
          docker cp convertToJSON.py "$ZIP_NAME:/work/convertToJSON.py"
          docker cp dashboard.py "$ZIP_NAME:/work/dashboard.py"
          docker cp seed_mongo.py "$ZIP_NAME:/work/seed_mongo.py"
          docker cp requirements.txt "$ZIP_NAME:/work/requirements.txt"
          docker cp Dockerfile "$ZIP_NAME:/work/Dockerfile"
          docker cp docker-compose.yaml "$ZIP_NAME:/work/docker-
compose.yaml"
          docker cp Jenkinsfile "$ZIP_NAME:/work/Jenkinsfile"
          docker cp README.txt "$ZIP_NAME:/work/README.txt"
          docker cp postman "$ZIP_NAME:/work/postman"
          docker cp data "$ZIP_NAME:/work/data"
          docker cp prometheus.yml "$ZIP_NAME:/work/prometheus.yml"
          docker cp grafana "$ZIP_NAME:/work/grafana"
          docker cp /tmp/make_zip.py "$ZIP_NAME:/work/make_zip.py"
```

```
          docker start -a "$ZIP_NAME"
          rm -rf artifacts
          docker cp "$ZIP_NAME:/work/artifacts" artifacts
          docker rm -f "$ZIP_NAME" >/dev/null 2>&1 || true
        '''
      }
    }
  }

  post {
    always {
      sh '''
        set +e
        NETWORK="ci-${BUILD_NUMBER}"
        MONGO_NAME="mongo-ci-${BUILD_NUMBER}"
        API_NAME="api-ci-${BUILD_NUMBER}"

        docker rm -f "$API_NAME" >/dev/null 2>&1 || true
        docker rm -f "$MONGO_NAME" >/dev/null 2>&1 || true
        docker network rm "$NETWORK" >/dev/null 2>&1 || true
      '''

      // Make outputs easy to download from Jenkins
      archiveArtifacts artifacts: 'README.txt,newman-
report.xml,artifacts/complete-*.zip', fingerprint: true, allowEmptyArchive:
true

      echo 'Pipeline complete.'
    }
  }
}
```

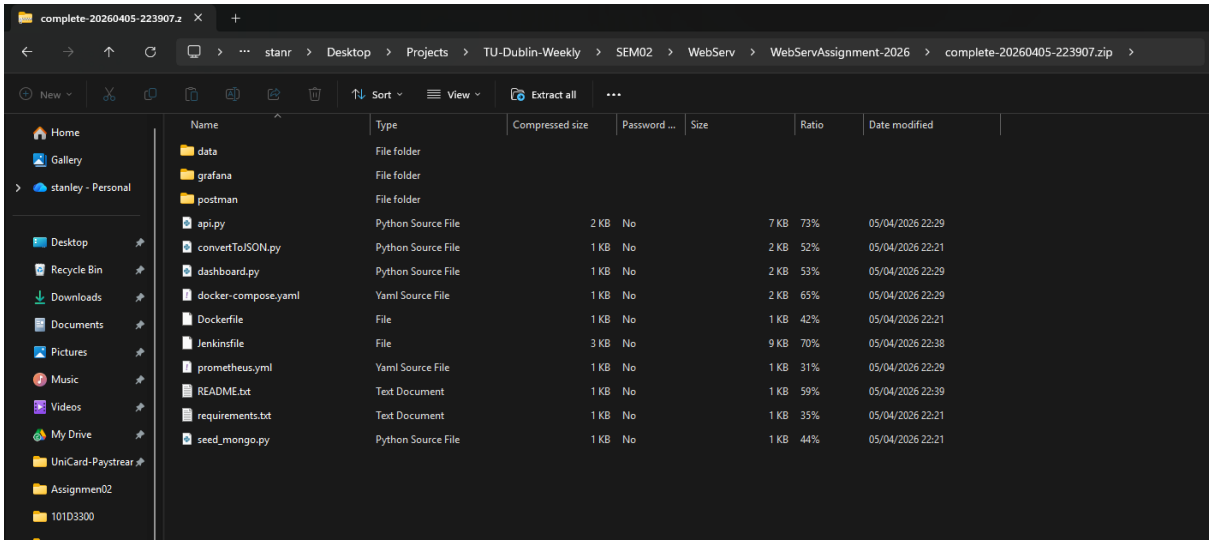
7. *Readme.txt*



8. *Newman Report:*


```
<?xml version="1.0" encoding="UTF-8"?>
<testsuites name="Inventory API - Endpoint Tests" tests="10" time="0.146">
  <testsuite name="Home" id="1c5d7897-3e29-4af1-9ec6-97cc2d71835e" timestamp="2026-04-05T22:39:05.730Z" tests="2" failures="0" errors="0" time="0.049">
    <testcase name="Status 200" time="0.049" classname="InventoryApiEndpointTests"/>
    <testcase name="Has endpoints list" time="0.049" classname="InventoryApiEndpointTests"/>
  </testsuite>
  <testsuite name="Get All Products" id="a3359060-8870-483c-86a5-dcd378645a67" timestamp="2026-04-05T22:39:05.730Z" tests="2" failures="0" errors="0" time="0.013">
    <testcase name="Status 200" time="0.013" classname="InventoryApiEndpointTests"/>
    <testcase name="Returns an array" time="0.013" classname="InventoryApiEndpointTests"/>
  </testsuite>
  <testsuite name="Get Single Product (1001)" id="35219814-b557-4a90-a715-85ffb39eec26" timestamp="2026-04-05T22:39:05.730Z" tests="2" failures="0" errors="0" time="0.004">
    <testcase name="Status 200" time="0.004" classname="InventoryApiEndpointTests"/>
    <testcase name="ProductID is 1001" time="0.004" classname="InventoryApiEndpointTests"/>
  </testsuite>
  <testsuite name="Starts With (S)" id="dd523fc5-269a-47fa-9298-93b1f38297cb" timestamp="2026-04-05T22:39:05.730Z" tests="2" failures="0" errors="0" time="0.003">
    <testcase name="Status 200" time="0.003" classname="InventoryApiEndpointTests"/>
    <testcase name="Returns array" time="0.003" classname="InventoryApiEndpointTests"/>
  </testsuite>
  <testsuite name="Paginate (1001-1010)" id="095166cb-d6a0-4f78-b2be-a60800b1101d" timestamp="2026-04-05T22:39:05.730Z" tests="2" failures="0" errors="0" time="0.003">
    <testcase name="Status 200" time="0.003" classname="InventoryApiEndpointTests"/>
    <testcase name="Returns array" time="0.003" classname="InventoryApiEndpointTests"/>
  </testsuite>
  <testsuite name="Convert (1001 USD->EUR)" id="0a3b5775-1761-46cb-a5a1-34e4813ea4db" timestamp="2026-04-05T22:39:05.730Z" tests="2" failures="0" errors="0" time="0.061">
    <testcase name="Status 200" time="0.061" classname="InventoryApiEndpointTests"/>
    <testcase name="Has PriceEUR" time="0.061" classname="InventoryApiEndpointTests"/>
  </testsuite>
  <testsuite name="Add New Product (9998)" id="39d64242-e846-4836-b562-d8d2cbd16a7d" timestamp="2026-04-05T22:39:05.730Z" tests="2" failures="0" errors="0" time="0.004">
    <testcase name="Status 200" time="0.004" classname="InventoryApiEndpointTests"/>
    <testcase name="Added successfully" time="0.004" classname="InventoryApiEndpointTests"/>
  </testsuite>
  <testsuite name="Get Single Product (9998)" id="613aaa03-dc01-409f-99ea-cb4f43aacf2a" timestamp="2026-04-05T22:39:05.730Z" tests="2" failures="0" errors="0" time="0.003">
    <testcase name="Status 200" time="0.003" classname="InventoryApiEndpointTests"/>
    <testcase name="ProductID is 9998" time="0.003" classname="InventoryApiEndpointTests"/>
  </testsuite>
  <testsuite name="Delete Product (9998)" id="c267cb86-55c2-4326-93fb-bcb8183ee850" timestamp="2026-04-05T22:39:05.730Z" tests="2" failures="0" errors="0" time="0.003">
    <testcase name="Status 200" time="0.003" classname="InventoryApiEndpointTests"/>
    <testcase name="Deleted successfully" time="0.003" classname="InventoryApiEndpointTests"/>
  </testsuite>
  <testsuite name="Get Deleted Product (9998) -> 404" id="62c3508c-d6f1-4ee2-bb9f-d71b286e2ca4" timestamp="2026-04-05T22:39:05.730Z" tests="1" failures="0" errors="0" time="0.003">
    <testcase name="Status 404" time="0.003" classname="InventoryApiEndpointTests"/>
  </testsuite>
</testsuites>
```


9. *Zip File Content*



Notes

Apendix

Docker:

Activation: docker compose up -d --build

Exit: docker compose down -v

Seed Script: docker compose exec api python3 seed_mongo.py

Exchange Rate:

exchange rate api: https://app.exchangerate-api.com/dashboard/confirmed

key: 9a503778311f71367cca243f

Graphana

url: http://localhost:3000

id: admin

pass: &gPpz@WfxSnWe6Dt*8XBH&p!T

References:

1. background Jenkins tasks: https://www.jenkins.io/doc/book/pipeline/docker/
2.